

YOLO



진행 상태



진행 중

Abstract

기존의 Object detection은 Classifier을 변형하여 탐지했다.

논문에서는 바운딩 박스와 그에 대응하는 클래스 확률을 이용하는 회귀 문제로 재정의 했다.

Introduction

인간은 이미지를 잠깐 보기만 해도 어떤 객체가 있는지, 어디에 위치하는지 등의 정보들을 순간적으로 인식할 수 있다.

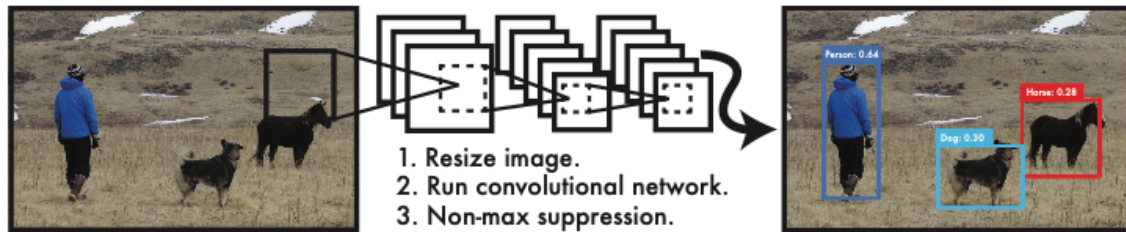
만약 인간과 같이 빠르고 정확한 알고리즘이 있다면 다양한 방면에서 활용할 수 있을 것이다.

현재의 객체 탐지 시스템은 Classifier을 변형하여 탐지한다.

DPM(Deformable Parts Model)은 sliding window 방식을 이용해 분류기를 이미지 전체의 일정한 위치에서 반복적으로 실행한다.

R-CNN은 region proposal 기법을 이용하여 이미지 내에서 잠재적인 바운딩 박스를 생성한 후 각각의 박스들에 대해 분류기를 실행한다. 그 후 후처리를 통해 중복되어 탐지된 부분은 제거하고, 장면 내에서 다른 객체를 고려하며 점수를 매긴다.

다만 이러한 방식은 파이프라인이 복잡하기 때문에 속도가 느리고 각 요소를 개별적으로 학습해야 하므로 최적화하기 어렵다.



YOLO는 이미지의 픽셀에서 바운딩 박스와 클래스 확률을 예측하는 하나의 회귀 문제로 본다.

하나의 합성곱 신경망이 여러개의 바운딩 박스와 각 박스의 클래스 확률을 동시에 예측한다. 이러한 YOLO가 가지는 여러 장점이 있다.

1. 회귀 문제로 정의했기 때문에 매우 빠르다.
2. 다른 실시간 시스템들에 비해 mAP가 2배 이상 우수하다.(정확도가 높다)
3. 이미지를 global하게 본다.

기존 다른 Classifier 방식에 비해 전체 이미지를 한 번에 본다. 이를 통해 문맥적 정보를 파악할 수 있다.

4. 높은 일반화 능력을 갖추었다.

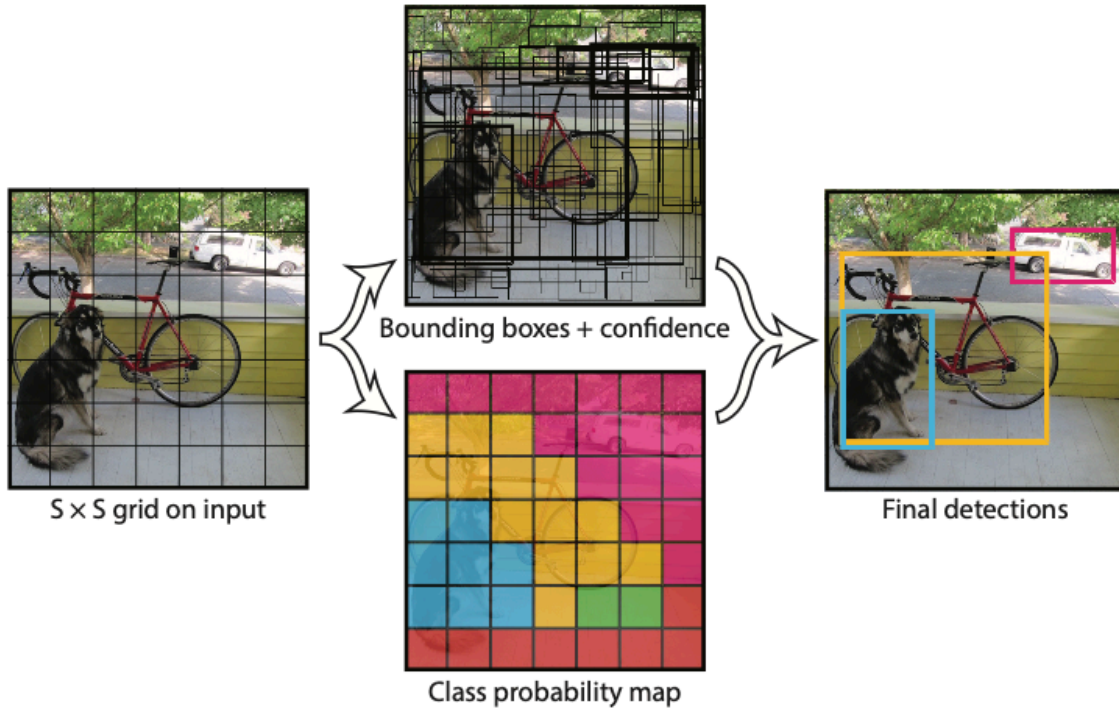
위와 같은 장점을 가졌지만 단점도 있다.

YOLO는 높은 속도를 가지고 있지만 정확도 측면에서 아직 SOTA에 비해 부족하다.

속도와 정확도는 Trade-off 관계로 보면 된다.

Unified Detection

YOLO가 어떤 방식을 이용하는지 알아보자



YOLO는 우선 입력 이미지를 $S \times S$ 그리드로 분할한다.

만약 객체의 중심이 특정 그리드 안에 존재하면 그 그리드가 객체 탐지의 책임을 갖는다.

그리고 각 그리드 셀은 B 개의 바운딩 박스와 그 박스에 대한 신뢰도(confidence score)를 예측한다.

신뢰도는 다음과 같다.

$$Confidence = Pr(Object) \times IOU_{pred}^{truth}$$

$Pr(Object)$: 박스에 객체가 존재한다고 확신하는 정도

IOU_{pred}^{truth} : ground truth와의 일치 정도 → 박스의 정확도

만약 객체가 박스에 존재하지 않으면 신뢰도는 0이어야하고,

객체가 존재한다면 신뢰도는 예측된 박스와 실제 박스 간의 IOU 값과 같아야 한다.

각 바운딩 박스는 5가지 예측값으로 구성된다.

(x, y) : 박스 중심 좌표(그리드 셀 기준)

w, h : 전체 이미지에 대한 상대적 폭과 높이

Confidence : 신뢰도

그리고 각 그리드 셀을 C개의 조건부 클래스 확률

$$Pr(Class_i|Object)$$

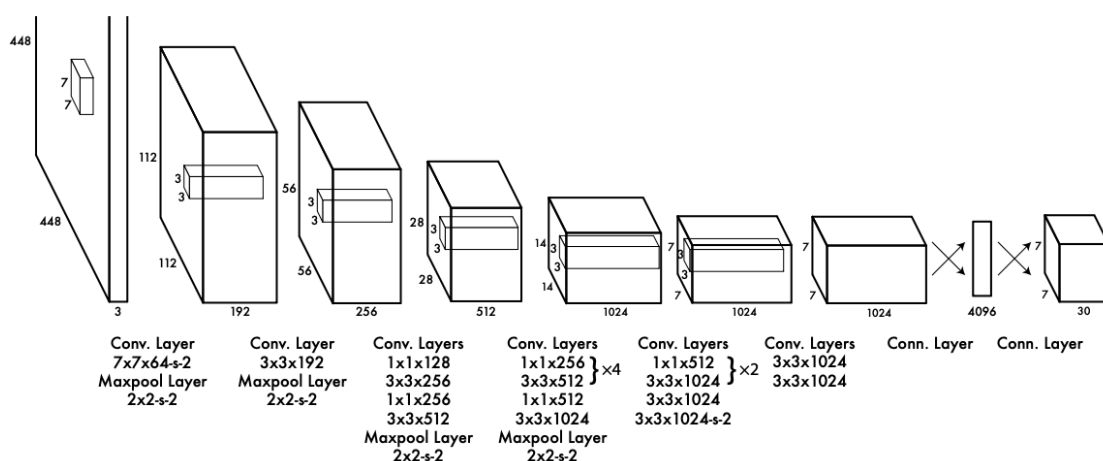
을 예측한다. (셀 안에 객체가 존재할 때)

그냥 객체가 존재할 때 그 객체가 무엇인지 클래스 확률을 예측한다고 보면 된다.

조건부 클래스 확률과 신뢰도를 곱하여 클래스별 confidence score 계산한다.

$$Pr(Class_i|Object) \times Confidence$$

Network Design



GoogLeNet에서 영감을 받아 24개의 합성곱 계층과 2개의 fully connected 계층으로 구성되어 있다.

또한, GoogLeNet의 inception module 대신 1X1 reduction layer, 3X3 합성곱 계층을 사용하였다.

객체 탐지의 속도를 시험하기 위해 Fast YOLO 모델을 사용하였다. 모델을 더 경량화시키기 위해

위 YOLO의 24개의 합성곱 계층을 더 줄인 9개의 계층을 사용하고, 필터 수도 더 적다.

Training

우선 처음 합성곱 계층 20개에 Avg Pool, FC 레이어를 추가한다.

ImageNet 1000 class 데이터셋을 이용해 학습하고 학습이

완료된 후 이를 Detection Task에 맞게 변환하기 위해 Conv 4개 + FC 2개 계층을 추가로 붙인다.

또한 탐지 작업은 세밀한 시각정보가 필요하기 때문에 입력 해상도를 224×224에서 448×448로 증가시킨다.

- 바운딩 박스의 폭과 높이는 이미지의 폭과 높이로 정규화 해서 0~1 값으로 설정
- 바운딩 박스의 x, y 좌표는 특정 그리드 셀 위치의 offset으로 매개변수화 하여 0~1 값으로 설정

최종 계층에서는 Leaky ReLU 함수를 활성화 함수로 사용한다.

출력에 대해서는 SSE를 이용해 최적화했다.

다만 대부분의 이미지에서 많은 그리드 셀은 객체를 포함하지 않는다. 그러한 셀들은 confidence값이 0에 가까워지는데 이 때문에 객체가 존재하는 셀에서의 gradient를 압도해버릴 수 있다. 따라서

- $\lambda_{coord} = 5$ (좌표 예측의 Loss)
- $\lambda_{noobj} = 0.5$ (객체가 없는 박스의 confidence Loss)

와 같이 파라미터를 설정한다.

또한, SSE는 큰 박스와 작은 박스에 따른 오차를 동일 가중치로 처리한다. 하지만 실제로 큰 박스 오차가 더 적은 영향을 가진다. 따라서 작은 박스의 오차에 더 민감하게 반응하기 위해 w, h 를 $\sqrt{w}, \sqrt{h}$ 와 같이 제곱근을 이용하여 처리한다.

YOLO는 다음과 같은 손실 함수를 사용한다.

$$\begin{aligned}
L = & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} [(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbf{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}$$

다음과 같은 데이터 증강 기법을 사용한다.

- 원본 이미지 크기의 최대 20% 범위에서 랜덤 스케일링과 translation을 적용.
- HSV 색 공간에서 이미지의 노출과 채도를 최대 1.5배까지 무작위로 조정

Limitations of YOLO

- 각 그리드 셀마다 2개의 바운딩 박스, 1개의 클래스를 예측할 수 있기 때문에 서로 근접한 여러 작은 개체들은 탐지하기 어렵다.
- 훈련 데이터와 다른 종횡비, 배치를 가지는 개체에서 일반화 성능이 떨어진다.
- 여러번 다운 샘플링 하기 때문에 세밀한 위치 추정이 어렵다.

Comparison to Other Detection Systems

- **Deformable parts models**

파이프라인으로 여러 단계를 가지는 모델
YOLO는 하나의 CNN으로 모든 과정을 통합한다.

- R-CNN

Selective Search 이후 CNN으로 특징을 추출하고 SVM 분류, 박스 조정, NMS과정을 거친다. 이와 같이 파이프라인이 복잡해서 속도도 느리다. 박스의 개수는 YOLO 약 98개 Selective Search 약 2000개로 큰 차이가 나고 YOLO는 공간적 제약 덕분에 중복 탐지를 완화할 수 있다.

- Fast R-CNN

R-CNN의 속도 문제를 어느정도 개선했지만 여전히 실시간 성능은 불가능하다.

대부분의 탐지 시스템과 비교해보았을 때

YOLO는 여러 객체, 클래스를 동시에, 그리고 범용으로 한 번에 처리하는 특징을 보였다.

Experiments

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Comparison to Other Real-Time Systems

YOLO는 PASCAL VOC 2007에서 다른 실시간 탐지기들과 비교했을 때, 속도와 정확도 면에서 모두 우위를 보였다. Fast YOLO는 52.7% mAP를 기록하며 기존 실시간 탐지기보다 두 배 이상 정확했다.

YOLO는 이를 한 단계 더 끌어올려 63.4% mAP를 달성하면서도 실시간 성능을 유지했다.

반면 Fast R-CNN, Faster R-CNN 등은 높은 정확도를 달성했지만 실시간 수준의 속도를 가지지는 못했다.

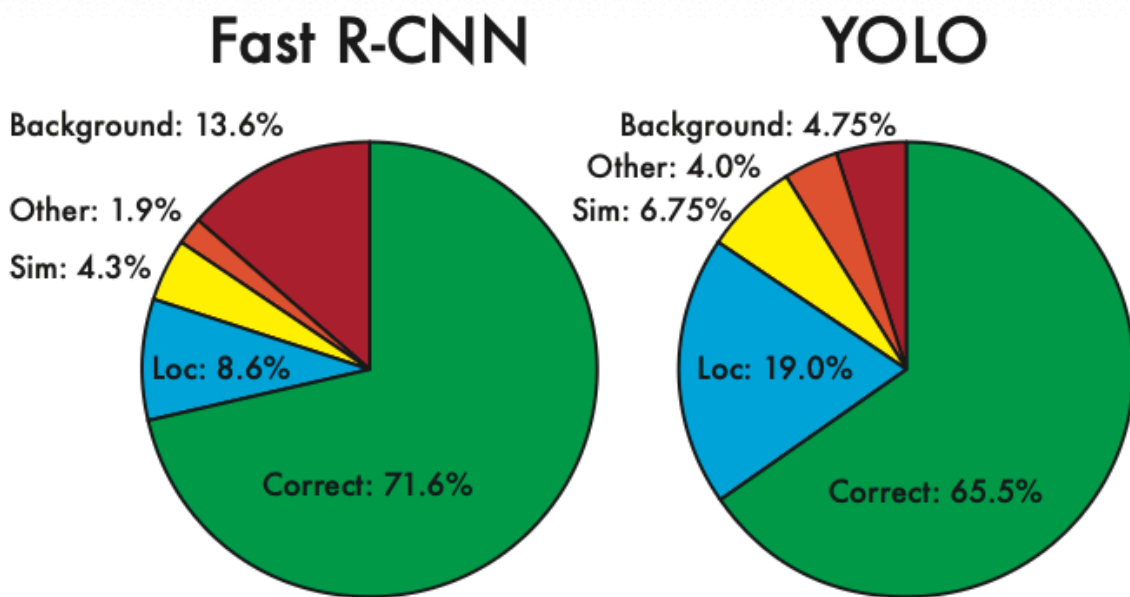


Figure 4: Error Analysis: Fast R-CNN vs. YOLO These charts show the percentage of localization and background errors in the top N detections for various categories (N = # objects in that category).

VOC 2007 Error Analysis

YOLO와 Fast R-CNN의 오류 패턴은 크게 달랐다. YOLO는 객체 위치를 정확히 잡지 못하는 Localization 오류가 많았다. 반면 Fast R-CNN은 Localization은 강했지만, 객체가 없는 배경을 잘못 인식하는 Background 오류가 많았다.

- **YOLO** → Localization 오류 비중이 가장 큼
- **Fast R-CNN** → Background 오류가 YOLO보다 약 3배 많음

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 data)	66.9	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	75.0	3.2

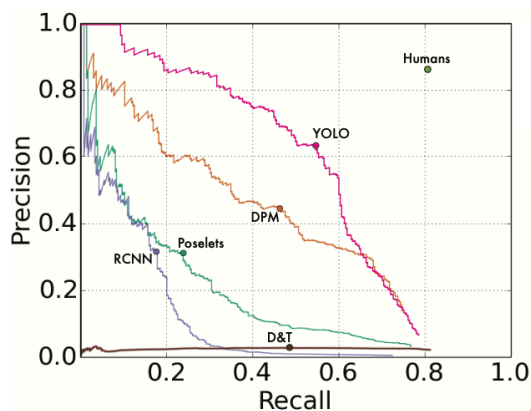
Combining Fast R-CNN and YOLO

두 모델을 결합했을 때는 성능을 향상시킬 수 있었다. Fast R-CNN의 박스를 YOLO 결과와 비교하여 겹치는 경우 YOLO의 확률을 반영해 보정했을 때, Fast R-CNN의 성능은 mAP 71.8%에서 75.0%로 약 3.2% 상승했다.

VOC 2012 Results

VOC 2012에서 YOLO는 57.9% mAP를 기록했는데, 이는 최신 모델보다는 낮은 수치다. 특히 작은 객체 탐지에서 어려움을 보였다.

예를 들어 bottle, sheep, tv/monitor 같은 클래스에서는 R-CNN보다 8~10% 낮았다. 하지만 cat, train과 같은 클래스에서는 오히려 더 높은 성능을 냈다.



(a) Picasso Dataset precision-recall curves.

	VOC 2007 AP	Picasso AP	Picasso Best F_1	People-Art AP
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Generalizability: Person Detection in Artwork

R-CNN은 자연 이미지에 맞춰진 Selective Search와 작은 영역 중심의 분류 구조 때문에 예술 task에서는 성능이 크게 저하되었다.

DPM은 낮은 정확도에도 불구하고 공간적 레이아웃 모델 덕분에 성능 저하가 적었다.

YOLO는 두 장점을 모두 가지며, VOC 2007에서도 높은 성능을 보였고 artwork로 도메인이 바뀌었을 때도 다른 탐지기보다 성능 저하가 적었다.